

Converted Image

Installing Flask and Setting the Home Route

Then install the Flask package and any dependencies.

Use the link to open `main.py`, so we can start coding the Flask backend for our app. Import the `Flask` package including `render_template` and `request`. Then import the `create_ascii` function from the `ascii_art` module. Create our app and the route for `/`. Render the `input.html` page when Flask loads. For now, create the `/ascii` route, but set the body of the function to `pass`. Finally, set the host to `0.0.0.0`, the port to `3000`, and debug to `True`.

```
from flask import Flask, render_template, request
from ascii_art import create_ascii

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('input.html')

@app.route('/ascii')
def ascii():
    pass

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=3000, debug=True)
```

```
python ascii-flask/main.py
```

[Open preview](#)

Because this function is interacting with the information in the `url` form, be sure to set `post` as an allowable method for the `ascii` route. Use `request.form` to take the information entered in the text box and store it in the `url` variable. Then create `ing` and set its value to the `create_ascii` function. Be sure to pass the user URL to this function. If `create_ascii` returns a list, create the response dictionary with the `'valid'` key set to `True` and the `'ing'` key set to `ing`.

If `create_ascii` does not return a list (the URL is not valid), create the response dictionary with `'valid'` set to `False` and `'img'` set to `img`. End the function by rendering the `art.html` template and pass it the response dictionary as `ascii`.

There is one big char that still needs to be made. Right now, our site will not render the ASCII art properly. That is because font in the terminal is a monospace font. That means every character has a fixed width. The default font for webpages are not monospace. As such, characters will have variable widths and the image will appear distorted. Open the `style.css` document and set the font family for the ASCII art to `monospace`. We will style the rest of the site on the next page.

```
#ascii-art {
  font-family: monospace;
}
```

Our Flask app is ready to run. Open the tab with the terminal and enter the command below.

```
python ascii-flask/main.py
```

Click the link to view the preview of the site. Use the link below as a test image for the webpage. You should see the same silhouette as before.

<https://images.pexels.com/photos/5302784/pexels-photo-5302784.jpeg?auto=compress&cs=tinysrgb&h=350&w=640&dpr=2>

Try these variations:

- Enter invalid image URLs to verify that the site properly displays a message to users.

```
conda deactivate
```

Assume the following code:

Assume the following code:

```
@app.route('/double')
def double():
    num = form['number']
    dbl = num * 2
    return render_template('new-number.html', new_num = dbl)
```

This route accepts a number, doubles it, and passes it to the `new-number.html` file as the variable `new_num`.

Select all of the things that are wrong with the aforementioned code. **Hint**, there is more than one correct answer.

- ☐ The route decorator should include `methods=["POST"]`.
- ☐ It should be `@App.route`.
- ☐ Use `request.form` instead of `form` when accepting a value from an HTML form.
- ☐ The return statement should be `template` instead of `render_template`.

The correct answers are:

- Use `request.form` instead of `form` when accepting a value from an HTML form.
- The route decorator should include `methods=("POST")`.

Flask's ability to accept data from is twofold. One, the route needs to be able to handle a `POST` request. Two, the `request` package contains `form`, which allows Flask to specify the name of the form from which the data comes.

Next